



第十一章内容回顾

www.shangqian.com.cn

QQ:1729366371

理解如何使用 C++ 面向对象的 I/O 流
能够格式化输入和输出
理解 I/O 流类的层次结构
理解怎样输入和输出用户自定义类型的对象
能够建立用户自定义的流操作符
能够确定流操作的成败





第十四章

智能指针



预习检查

www.shangqian.com.cn

QQ:1729366371

<1> 智能指针和普通指针的差异，为何要引入智能指针？

原始指针：通过 `new` 建立的 * 指针

智能指针：通过智能指针关键（`auto_ptr`）建立的指针。

通过智能指针能够实现普通指针实现不了的功能，可以通过模板类的成员函数 `get()` 返回一个用于传递普通指针，实现向下兼容。

智能指针能够自动回收资源，确保程序不存在内存何资源泄露切是异常安全的。通过 `new` 操作符需要自定义 `delete` 进行资源回收，存在这异常安全的隐患，造成内存泄露 / 野指针。在现代 C++ 中，原始指针仅用于范围有限的小代码块、循环或者性能至关重要且不会混淆所有权的 `Helper` 函数中。



本章目标

www.shangqian.com.cn

QQ:1729366371

1. 智能指针背后的设计思想

2. C++ 智能指针简单介绍





14.1 设计思路

www.shangqian.com.cn

QQ:1729366371

```
void remodel(std::string & str)
{
    std::string * ps = new std::string(str);
    ...
    throw exception();
    str = *ps;
    delete ps;
    return;
}
```



当出现异常时（**weird_thing()** 返回 **true**），**delete** 将不被执行，因此将导致内存泄露。

如何避免这种问题？有人会说，这还不简单，直接在 **throw exception();** 之前加上 **delete ps;** 不就行了。是的，你本应如此，问题是很多人都会忘记在适当的地方加上 **delete** 语句（连上述代码中最后的那句 **delete** 语句也会有很多人忘记吧），如果你要对一个庞大的工程进行 **review**，看是否有这种潜在的内存泄露问题，那就是一场灾难！

这时我们会想：当 **remodel** 这样的函数终止（不管是正常终止，还是由于出现了异常而终止），本地变量都将自动从栈内存中删除—因此指针 **ps** 占据的内存将被释放，如果 **ps** 指向的内存也被自动释放，那该有多好啊。



我们知道析构函数有这个功能。如果 **ps** 有一个析构函数，该析构函数将在 **ps** 过期时自动释放它指向的内存。但 **ps** 的问题在于，它只是一个常规指针，不是有析构函数的类对象指针。如果它指向的是对象，则可以在对象过期时，让它的析构函数删除指向的内存。

这正是 **auto_ptr**、**unique_ptr** 和 **shared_ptr** 这几个智能指针背后的设计思想。我简单的总结下就是：将基本类型指针封装为类对象指针（这个类肯定是个模板，以适应不同基本类型的需求），并在析构函数里编写 **delete** 语句删除指针指向的内存空间。



因此，要转换 **remodel()** 函数，应按下面 3 个步骤进行：
包含头文件 **memory** （智能指针所在的头文件）；
将指向 **string** 的指针替换为指向 **string** 的智能指针对象；
删除 **delete** 语句。

```
# include <memory>
```

```
void remodel (std::string & str)
```

```
{
```

```
    std::auto_ptr<std::string> ps (new std::string(str)) ;
```

```
    throw exception() ;
```

```
    str = *ps ;
```

```
    // delete ps ; NO LONGER NEEDED
```

```
    return;
```

```
}
```